

Array Basics and Operations

Subject: Data Structures Topic: Arrays

Overview

Comprehensive notes on array fundamentals, time complexity, and common operations

Array model and complexity

- An array stores values in contiguous memory; index i is at base address plus i multiplied by element size.
- Reading or replacing an element by index is $O(1)$. Searching an unsorted array is $O(n)$.
- Inserting or removing in the middle is $O(n)$ because later elements must shift.

Core operations

- Traversal visits each element once, so its time is $O(n)$ and extra space is $O(1)$.
- Linear search works on any array; binary search requires sorted data and halves the remaining range each step.
- Two-pointer scans can solve sorted pair and partition problems in $O(n)$ after sorting or when the input is already ordered.

Worked example

- For $[4, 7, 9, 12]$, index 2 contains 9; inserting 8 at index 2 shifts 9 and 12 to the right.
- To find 12 in a sorted array, binary search checks the middle, discards the lower half, and repeats; the search takes $O(\log n)$.
- Choose arrays when fast indexed access and cache locality matter more than frequent middle insertions.

Review checklist

- Check index bounds and empty-array cases before accessing an element.
- Distinguish an array's length from its final valid index, which is length minus one.
- State time and auxiliary-space complexity for each operation.